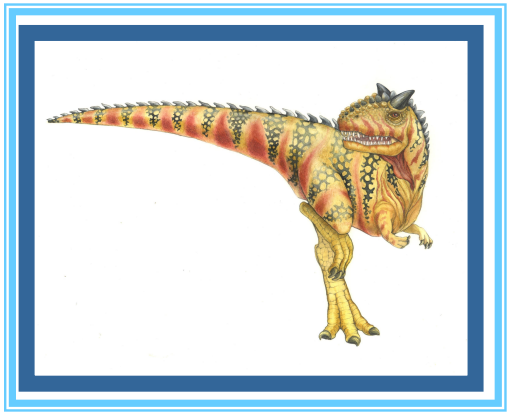
Chapter 3: Processes



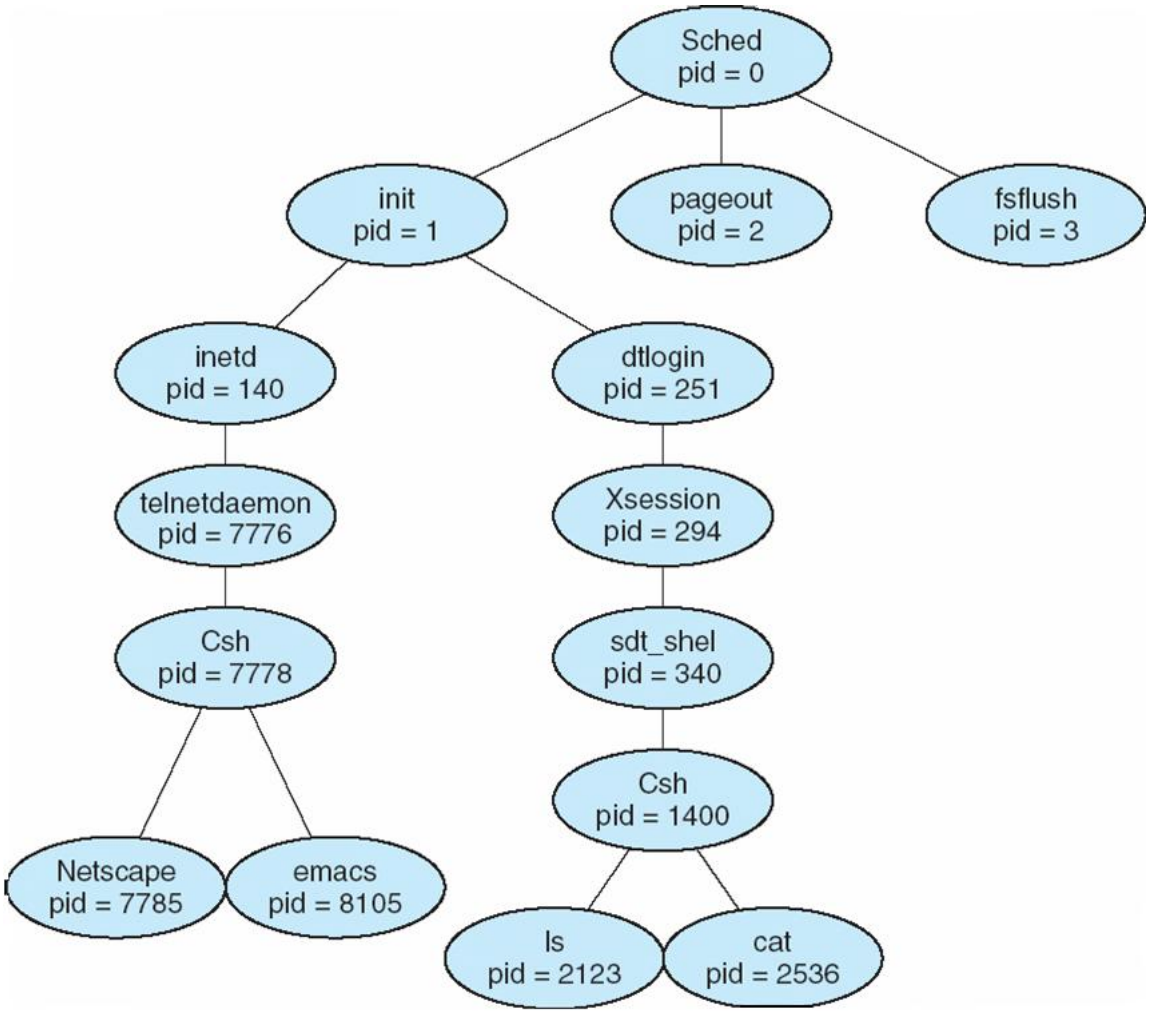
Operations on processes

- Process creation
- Process Termination
- Process blocking
- Process wakeup
- Process suspend
- Process activate

Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via **a process identifier** (**pid**)
- Resource sharing
  - Parent and children share all resources （嵌入式系统）
  - Children share subset of parent's resources （Unix系统）
  - Parent and child share no resources（Windows系统）
- Execution
  - Parent and children execute concurrently
  - Parent waits until children terminate

A tree of processes on a typical Solaris



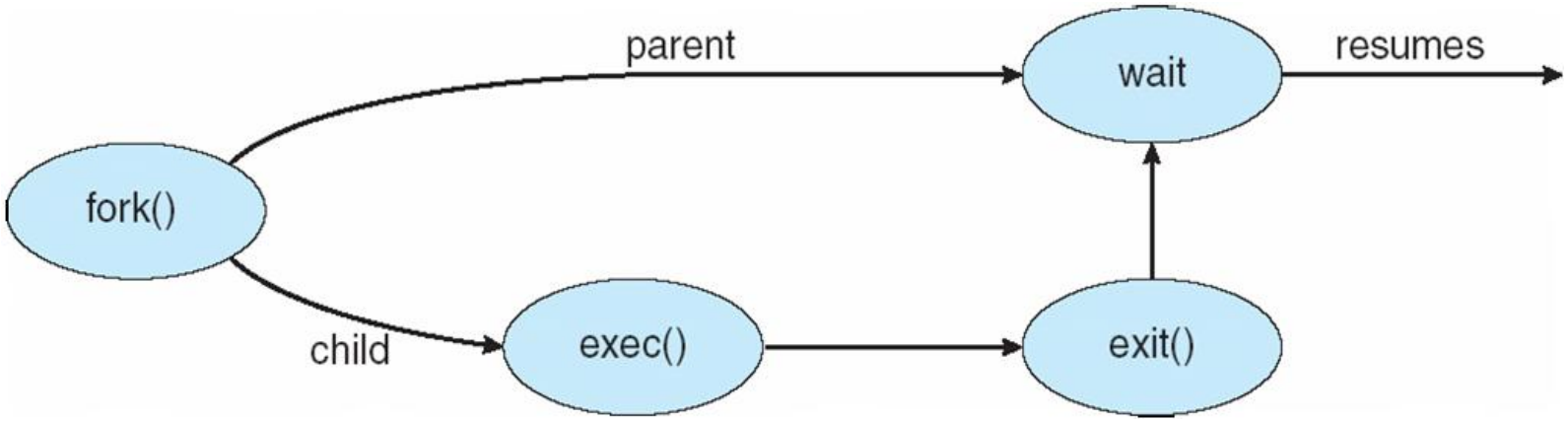
Process Creation (Cont)

- Address space （共享/独立）
  - Child duplicate of parent (Unix)
  - Child has a program loaded into it (Windows)
- UNIX examples
  - **fork** system call creates new process
  - **exec** system call used after a **fork** to replace the process' memory space with a new program
- Windows NT
  - Supports both models: the parent's address space may be duplicated, or the parent may specify the name of a program for the OS to load into the address of the new process

C Program Forking Separate Process

```
int main()
{
pid_t  pid;
   /* fork another process */
   pid = fork();
   if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
   }
   else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
   }
   else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
   }
}
```

Process Creation



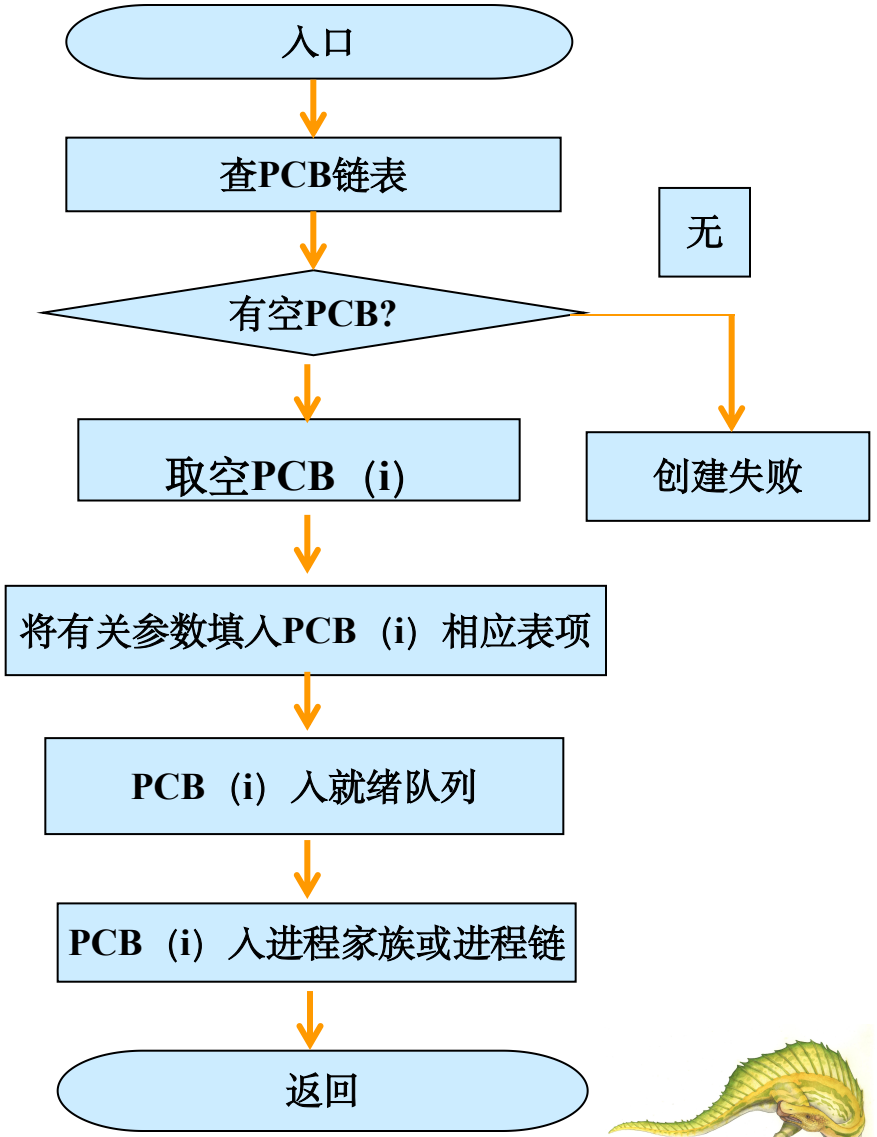
Process creation

- UNIX

  pid=fork();

  从系统调用fork返回

- 若在父进程时，pid值为所创建子进程的进程号（>0）
- 若在子进程时，pid的值为零



Process Creation

```
int main ( )
{
      while (1)
       {
           fork( );
        }
}
```

Process Creation

```
int main (void)
{
      int i;
       for (i=0; i<3; i++)
           fork ( );
       sleep (3);
}
```

C Program Forking Separate Process

```
int value;
int main ( ){
    pid_t  pid;
    pid = fork ( );
    if (pid < 0) { /* error occurred */
            fprintf (stderr, "Fork Failed");
            exit (-1);
    }
    else if (pid == 0) { /* child process */
             for ( ; ; ) {
                 printf ("In child, value is %d\n ", value++);
                sleep (2) ;
             }
    }
    else { /* parent process */
            for ( ; ; ) {
                printf ("In parent, value is %d\n ", value);
                sleep (2);
            }
    }
}
```

C Program Forking Separate Process

```
int value;
int main( ){
        pid_t  pid;
        pid = fork ( );
        if (pid < 0) { /* error occurred */
                fprintf (stderr, "Fork Failed");
                exit (-1);
        }
        else if (pid == 0) { /* child process */
                for ( ; ; ) {
                    printf ("In child, value is %p\n ", &value);
                    sleep(2) ;
                }
        }
        else { /* parent process */
                for ( ; ; ) {
                    printf ("In parent, value is %p\n ", &value);
                    sleep (2);
                }
        }
}
```

Process Creation

当父进程调用fork()创建子进程之后，下列哪些变量在子进程中修改之后，父进程里也会相应地作出改动？

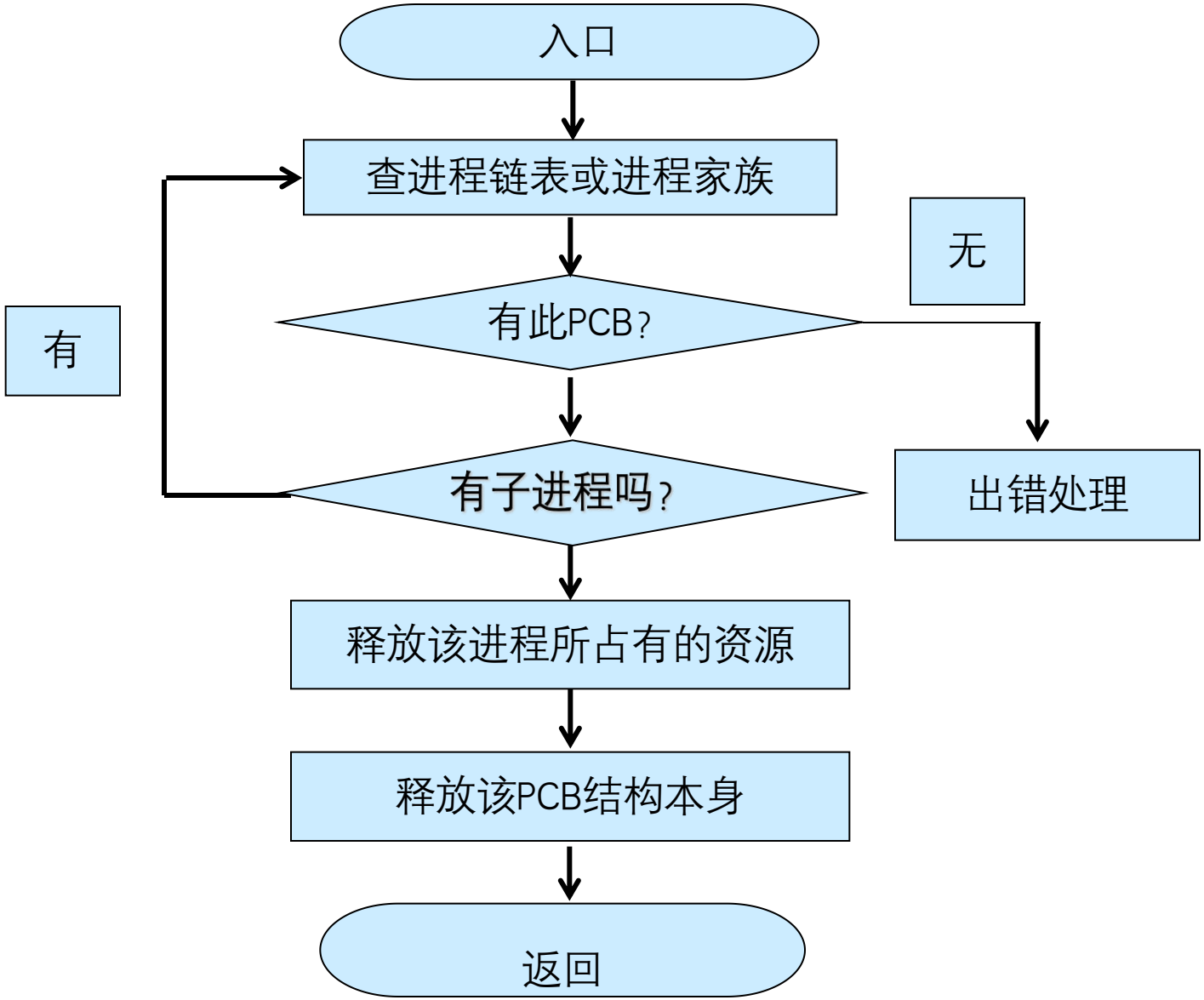
A.全局变量
B.局部变量
C.静态变量
D.文件指针

答案：D
在fork创建多进程之后堆栈信息会完全复制给子进程内存空间，父子进程相互独立。文件描述符存在于系统中为所有进程所共享

Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
  - Output data from child to parent (via **wait**)
  - Process' resources are deallocated by operating system
- Parent may terminate execution of children processes (**abort**)
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - Some operating system do not allow child to continue if its parent terminates
      - All children terminated - **cascading termination**

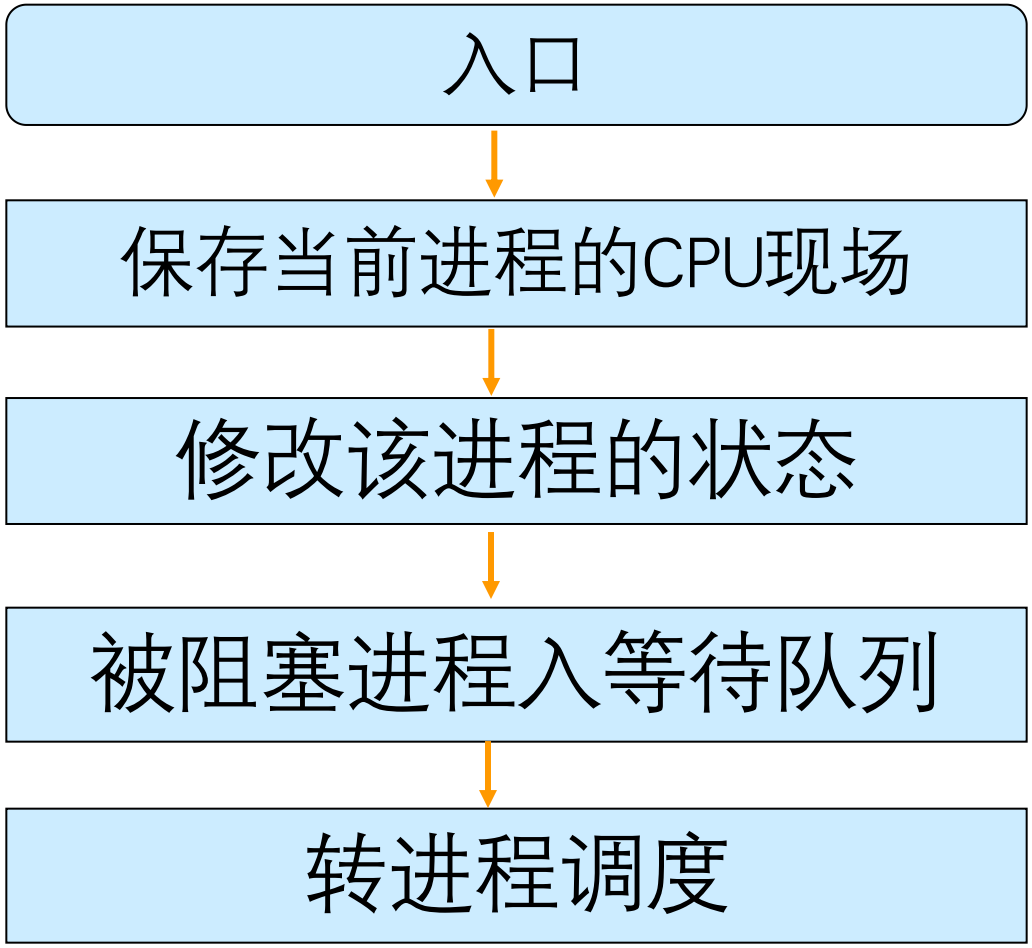
Process Termination



Process blocking and wakeup

- A running process can not continue to run because of something: Running → blocked
  - I/O
  - Signal
- When the event happens, the waiting process will be wakeup
  - blocked → ready

Process Blocking



Process wakeup

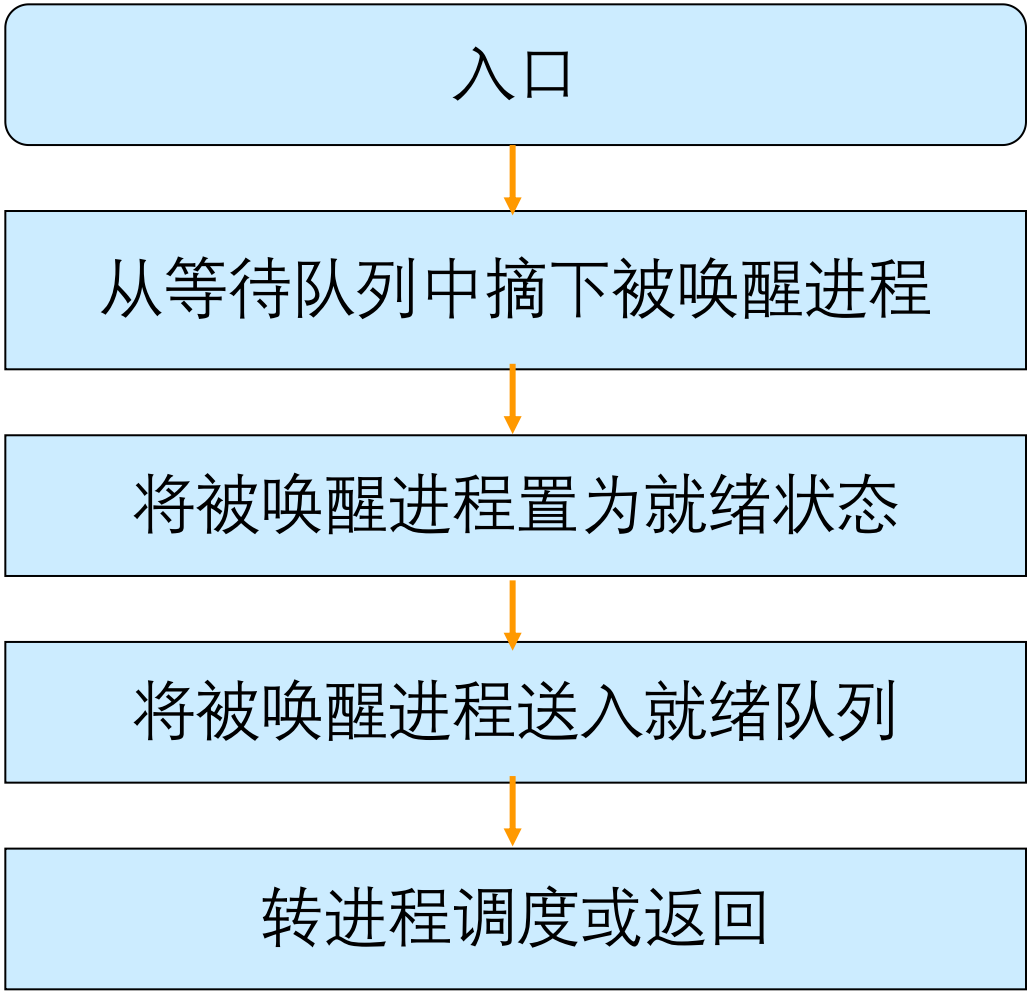
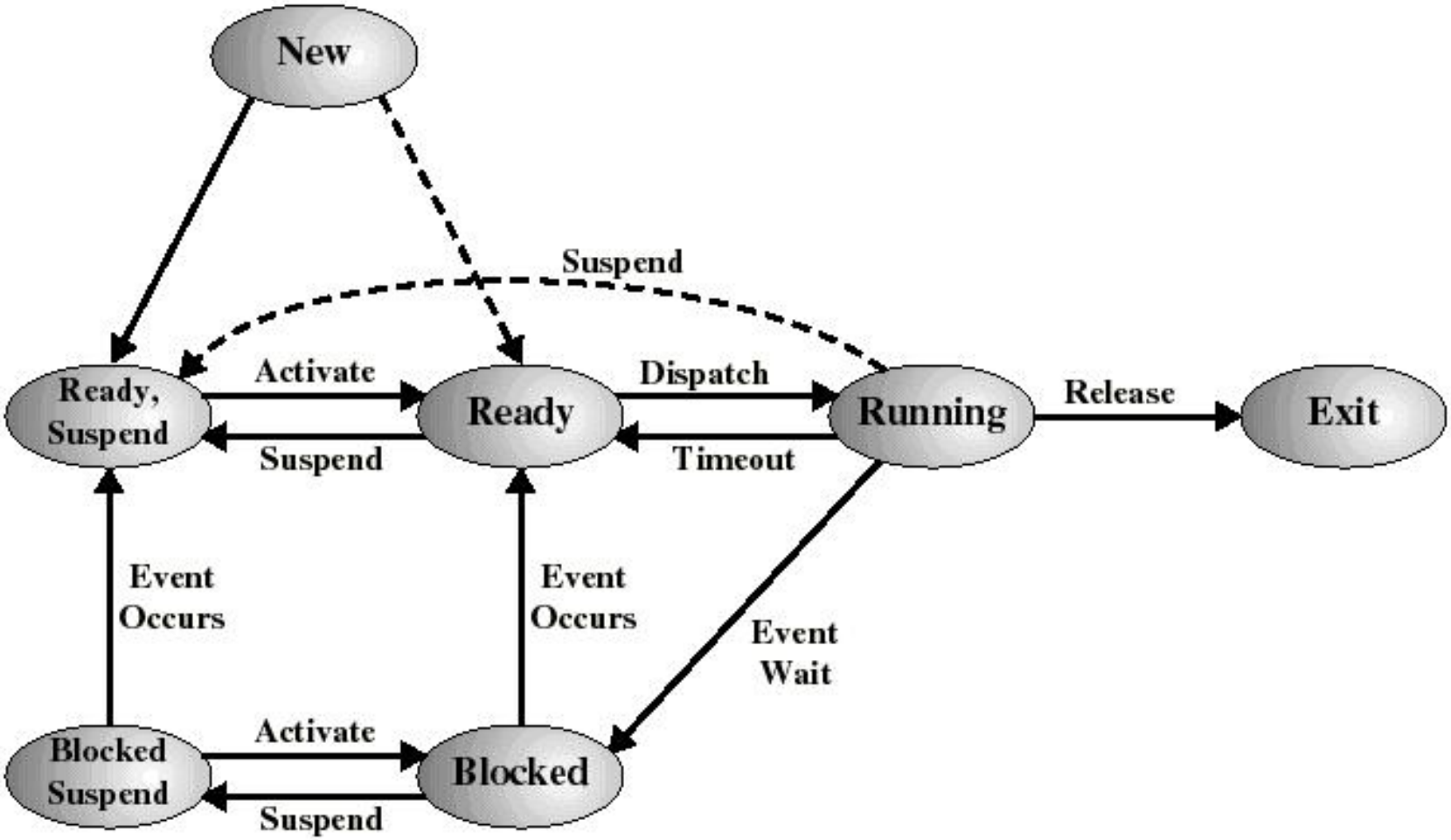


Diagram of Process State



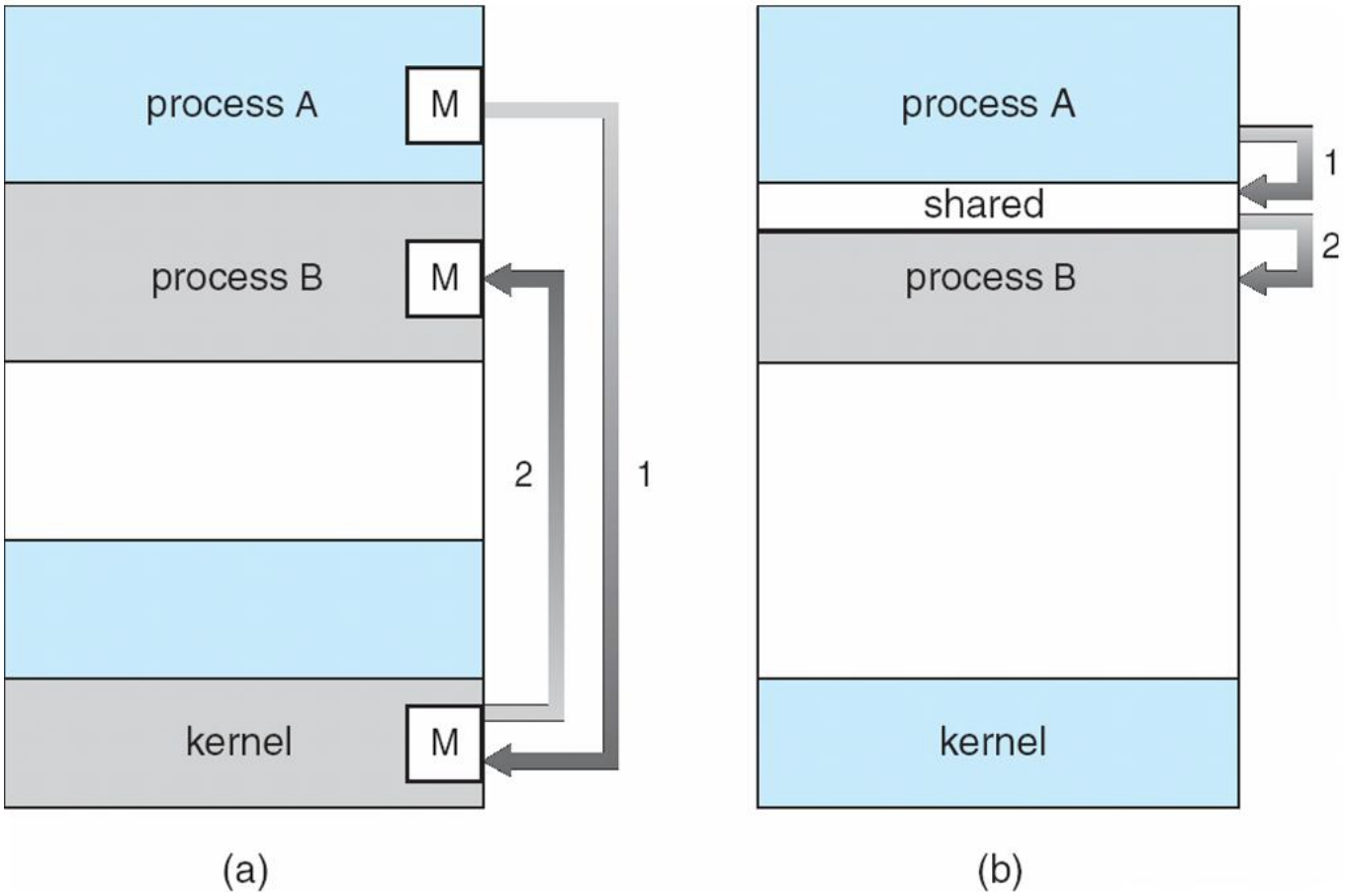
Interprocess Communication

- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience

Interprocess Communication

- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - Shared memory
  - Message passing

Communications Models



Shared memory

- Interprocess communication using Shared memory requires communicating processes to establish a shared memory
- The form of the exchanged data and location are determined by the communicating processes and are not under the OS's control

Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
  - *unbounded-buffer* places no practical limit on the size of the buffer
  - *bounded-buffer* assumes that there is a fixed buffer size

Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
   . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Bounded-Buffer – Producer

```
while (true) {
   /* Produce an item */
      while (((in = (in + 1) % BUFFER SIZE
count)  == out)
      ;   /* do nothing -- no free buffers */
    buffer[in] = item;
    in = (in + 1) % BUFFER SIZE;
   }
```

Bounded Buffer – Consumer

```
while (true) {
        while (in == out)
                ; // do nothing
-- nothing to consume

     // remove an item from
the buffer
     item = buffer[out];
     out = (out + 1) % BUFFER
SIZE;
return item;
   }
```

Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **send**(*message*) – message size fixed or variable
  - **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
  - establish a *communication link* between them
  - exchange messages via send/receive

Implementation Questions

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?

Message Passing

- Direct Communication
- Indirect Communication

Direct Communication

- Processes must name each other explicitly:
  - **send** (*P, message*) – send a message to process P
  - **receive**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional

Indirect Communication

- Operations
  - create a new mailbox
  - send and receive messages through mailbox
  - destroy a mailbox
- Primitives are defined as:

  send(*A, message*) – send a message to mailbox A

  receive(*A, message*) – receive a message from mailbox A

Indirect Communication

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

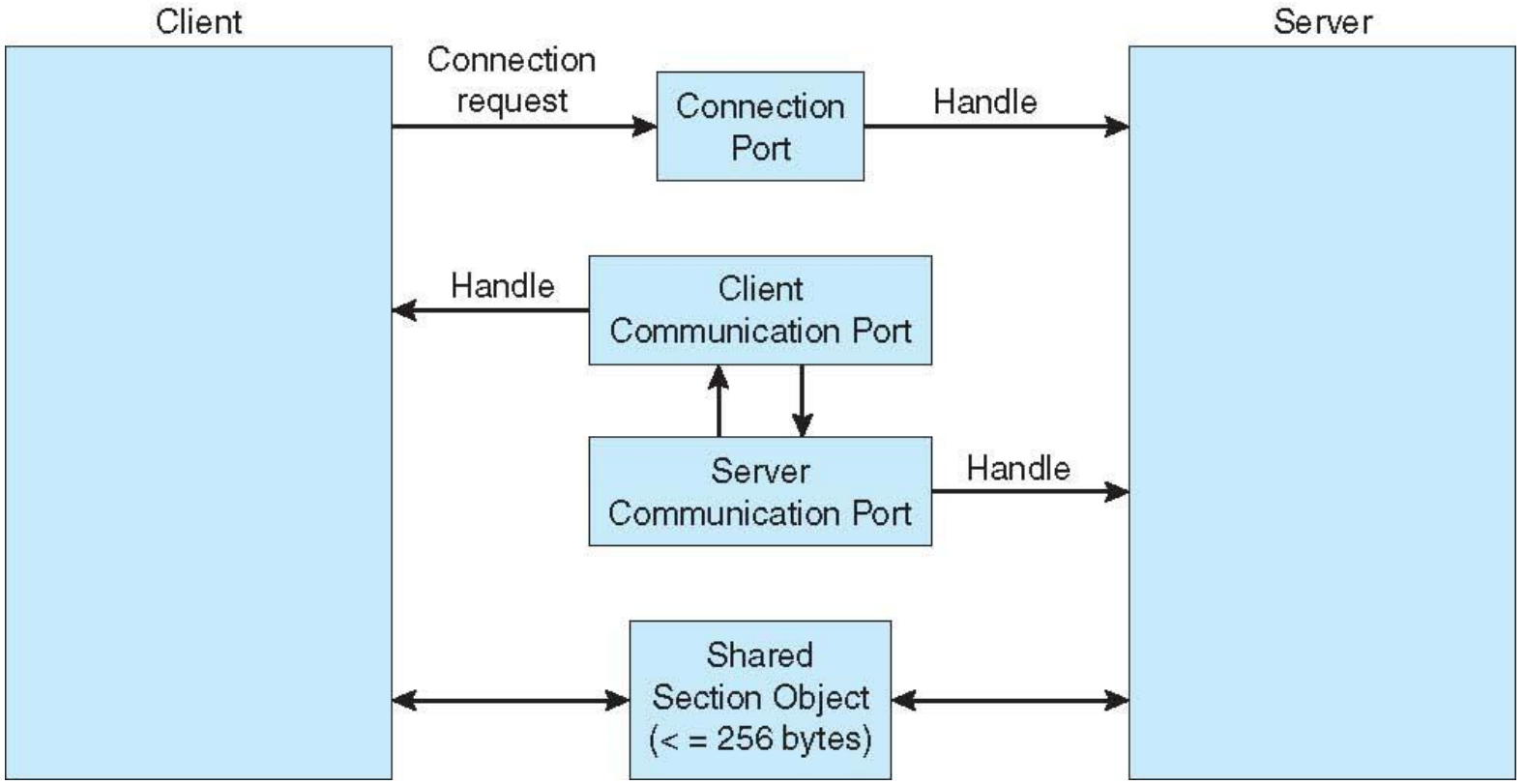
Message-passing----Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
  - **Blocking send** has the sender block until the message is received
  - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking** send has the sender send the message and continue
  - **Non-blocking** receive has the receiver receive a valid message or null

Buffering

- Queue of messages attached to the link; implemented in one of three ways
  1. Zero capacity – 0 messages
     Sender must wait for receiver (rendezvous)
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits

Local Procedure Calls in Windows XP



Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

End of Chapter 3